

Lab 0 — Computers Speak Vectors

Friday, August 28

Last updated: August 20, 2026.

In class Friday, August 28 (same day as [Lecture 03](#)).

Credit: Lab points are for effort in class, not for a Canvas file upload. Work the exercises in Octave or Python (bring a laptop; demos and both-language code are on this page). On lab day you get a hard copy with the exercises written out, space for answers and new syntax. Preview the [one-pager](#); write on the hard copy handed out in class.

Pick **Octave or Python** No prior coding is assumed.

Software: [GNU Octave](#), or Python in [Google Colab](#) (no local install). Links for Lachniet, NumPy, Polars, and Plotnine are on [Resources](#).

Learning objectives

1. Enter, index, and modify vectors in Octave or Python.
2. Perform vector arithmetic and create a basic plot.
3. Modify supplied code and explain its output or error message.

Math connections

Linear combinations and scaling from [Lecture 03](#); vector representation from [Lecture 01](#).

Key syntax (this lab)

Syntax roles: *name* = you chose the name; *value* = you chose the number; *lang* = built-in language word. (On the website these three roles are color-coded.)

Meaning	Boyd	Octave	Python
first entry of x	x_1	<code>x(1)</code>	<code>x[0]</code>
second entry	x_2	<code>x(2)</code>	<code>x[1]</code>
length n	n	<code>length(x)</code>	<code>len(x)</code>

Resources: Lachniet Ch. 1; [NumPy beginners](#); full list on [Resources](#).

Assignment vs equality

In math, `=` often means “these are equal.” In code, a single `=` almost always means **assignment**: “compute the right-hand side and store it under this name.”

```
n = length(x)    % Octave: store the length of x in the name n
```

```
n = len(x)       # Python: same idea
```

After that line runs, **n holds** a number. Checking whether two things are equal uses a different symbol later (`==` in both languages); you do not need that today.

Functions look like $f(x)$

A **function** has a name. You **call** it by writing the name, then parentheses, then an **input**.

Math idea	In words	Role in code
f	the function's name	often a language word such as <code>length</code> or <code>len</code>
x	the input you give it	often a name you chose such as <code>x</code>
$f(x)$	"do f to this input"	<code>length(x)</code>
$y = f(x)$	"store the result as y "	<code>n=length(x)</code>

So `length(x)` means: run the built-in named `length` on the object you named `x`. The result is a number (how many entries `x` has).

Where you type: Octave GUI

1. Install/open [GNU Octave](#) (the GUI is enough).
2. Use the **Command Window** to type one line at a time and press Enter. Octave prints results below.
3. Optional: open a new **Editor** tab, paste several lines (a short script), save as something like `lab00.m`, and run it. Same commands; easier to edit and re-run.
4. If a plot window appears, leave it open while you answer the ticket.

Lachniet Ch. 1 walks through the interface in more detail.

Where you type: Google Colab (Python)

1. Open [Google Colab](#), sign in, and start a new notebook (**File** → **Save a copy in Drive** so you keep your work).
2. A notebook mixes **text cells** (notes, like this page) and **code cells** (Python you run).
3. Click a code cell, paste a demo, then run it with the play button or Shift+Enter. **Output** appears directly under that cell (numbers, errors, or plots).
4. Run cells **in order** from top to bottom the first time. If something breaks, read the error under the cell and ask in lab.

Indexing

Boyd writes x_1 for the first entry. Octave counts from **1**. Python/NumPy count from **0**.

The data

Use this 6-vector of **Arcata morning temperatures**, in °F, for six successive hours:

$$x = (52, 51, 53, 55, 58, 60).$$

Entry x_1 is 7 a.m.; x_6 is noon. Work **C0.1–C0.6 in order**.

C0.1 — Enter and index

Create a vector named `x`. Print `n`, the first entry, the last entry, and Boyd's third entry x_3 .

Octave (demo — run this):

```
x = [52, 51, 53, 55, 58, 60];
n = length(x)
x(1)
x(n)
x(3)
```

Python (demo — run this):

```
import numpy as np
x = np.array([52, 51, 53, 55, 58, 60])
n = len(x)
print(n, x[0], x[-1], x[2])
```

Expected output (values): $n = 6$, first entry 52, last entry 60, third entry 53.

Check What quantity does x_3 represent (include units)? Write the command that prints that entry.

C0.2 — Modify two entries

The 9 a.m. reading was mistyped: it should be 54 instead of 53. The noon reading should be 61 instead of 60. Change those two entries of **x**. Print the new vector.

Expected vector: (52, 51, 54, 55, 58, 61).

Hints:

```
x(3) = 54;
x(6) = 61;
x
```

```
x[2] = 54
x[5] = 61
print(x)
```

Check: Record your new 6-vector.

C0.3 — Scale

Using your **corrected x** from C0.2, create a vector named **doubled** equal to $2*x$. Print **doubled**.

Expected output: (104, 102, 108, 110, 116, 122).

Check: Write the assignment line you used. What does multiplying by 2 mean for these temperatures?

C0.4 — Add a vector

Create a vector named **ones6** of six ones (every entry 1). Then create **warmer** by adding five degrees to every hour: **warmer**=**x**+5***ones6**. Print **warmer**.

Expected output: (57, 56, 59, 60, 63, 66).

Hints for **ones6**: Octave **ones(1,6)** or **[1,1,1,1,1,1]**; Python **np.ones(6)** or **np.array([1,1,1,1,1,1])**.

Check: Record **warmer**. Why is **x+5** (adding a scalar) not the same idea in the mathematics of this course, even if software accepts it?

C0.5 — Plot x , $2x$, and $-x$

Plot the corrected x , $2x$, and $-x$ versus hour index $i = 1, \dots, 6$. Include a legend.

Octave:

```
i = 1:6;
plot(i, x, '-o', i, 2*x, '-o', i, -x, '-o');
legend('x', '2x', '-x');
xlabel('hour index i');
ylabel('value');
```

Python:

```
import matplotlib.pyplot as plt
i = np.arange(1, 7)
plt.plot(i, x, '-o', label='x')
plt.plot(i, 2*x, '-o', label='2x')
plt.plot(i, -x, '-o', label='-x')
plt.xlabel('hour index i')
plt.ylabel('value')
plt.legend()
plt.show()
```

Check: In one sentence, what did multiplying by -1 do in the picture?

C0.6 — Predict, then check

Before running anything new, **predict** what the following prints. Then type it and compare.

Octave:

```
v = [10, 20, 30];
v(2)
```

Python:

```
v = np.array([10, 20, 30])
print(v[1])
```

Expected output: 20 (Boyd's second entry v_2).

Check: Your prediction before running: _____. Actual output: _____. In Boyd indexing, that entry is v _____.